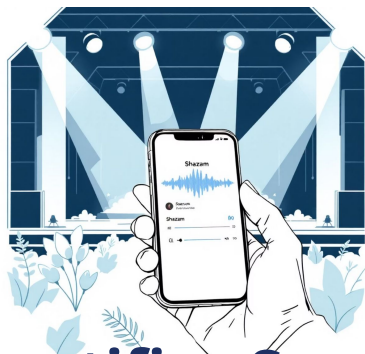




How Shazam Identifies Songs in 5 Seconds: Audio Fingerprinting with Python

Özge Çinko

PyCon Netherlands 2025



Özge Çinko

AI Research Engineer at Huawei



Contents

- The Magic Behind Shazam
- From Sound Waves to Spectrograms
- Finding What Makes a Song Unique
- Building Audio Fingerprints
- Matching Songs in the Database
- Let's Build It: A Mini Shazam in Python
- Real World Challenges
- Beyond Music Recognition
- Question & Answer

The Magic Behind Shazam

What is Shazam?

A music recognition app that listens for just a few seconds and gets it right.

2002

The Phone Call Era



2008

App Store Era



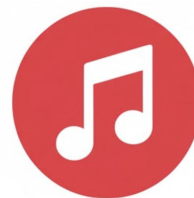
2012

Beyond Music



Today

Part of the Apple Family



The Magic Behind Shazam

The “Magic” Behind Isn’t Magic

It’s robust engineering against all odds.



Capture & Simplify

Create a Spectrogram and Find the Fingerprints

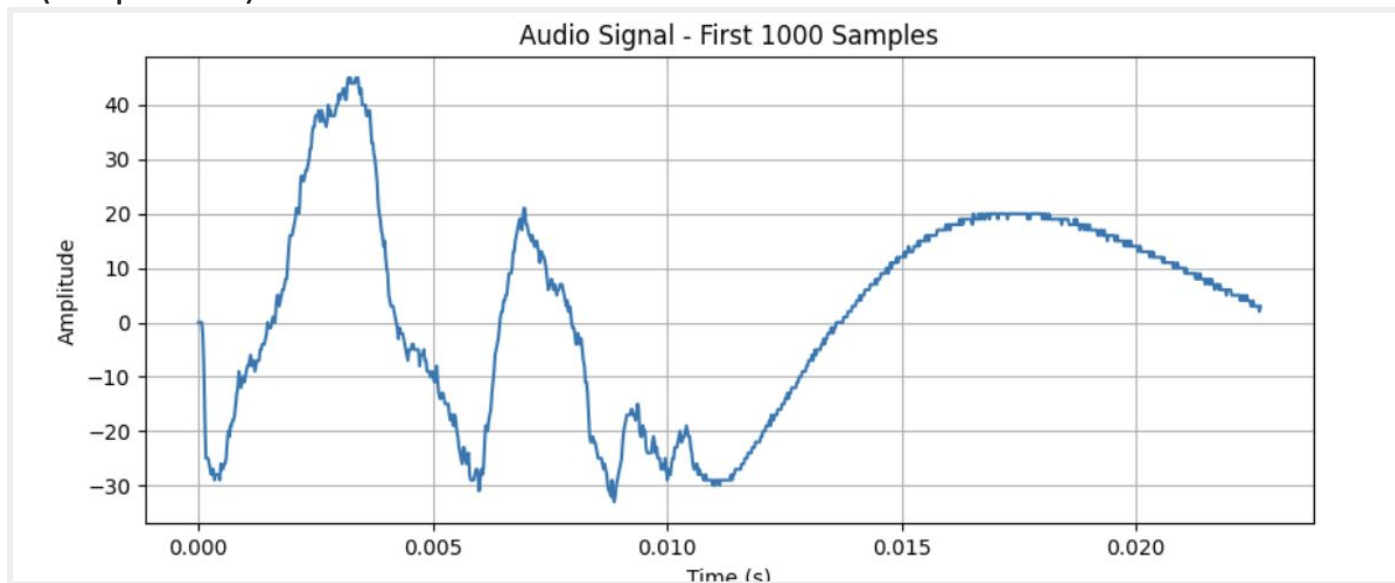
The Lightning-Fast Search

From Sound Waves to Spectrogram

Capturing The Sound Wave

Here's discrete digital samples of my "Hello World"!

- X-axis (Time): *When* the sound happens
- Y-axis (Amplitude): *How loud* it is at that moment



From Sound Waves to Spectrogram

How to Capture A Sound Wave?

First, we need to set three parameters:

- **Sampling Rate (44.1 kHz)** - How often we measure the sound wave
- **Bit Depth (16-bit)** - How detailed each measurement is
- **Channel (Mono)** - How many audio tracks we record

From Sound Waves to Spectrogram

Capture The Sound Wave with Python

```
import numpy as np
import matplotlib.pyplot as plt
import sounddevice as sd

fs = 44100      # Sampling rate (44.1 kHz)
duration = 3    # Record for 3 seconds
bit_depth = 16  # 16-bit quantization

audio = sd.rec(int(duration * fs), samplerate=fs, channels=1, dtype='int16')
sd.wait()

samples = audio[:1000].flatten()
time = np.arange(len(samples)) / fs
plt.plot(time, samples) # first 1000 samples
```

Sampling Rate: 44.1 kHz
Quantization: 16-bit
Channel: Mono

From Sound Waves to Spectrogram

Why **sounddevice**?



```
pip install sounddevice
```

spatialaudio/**python-
sounddevice**

🔊 Play and Record Sound with Python 🐍

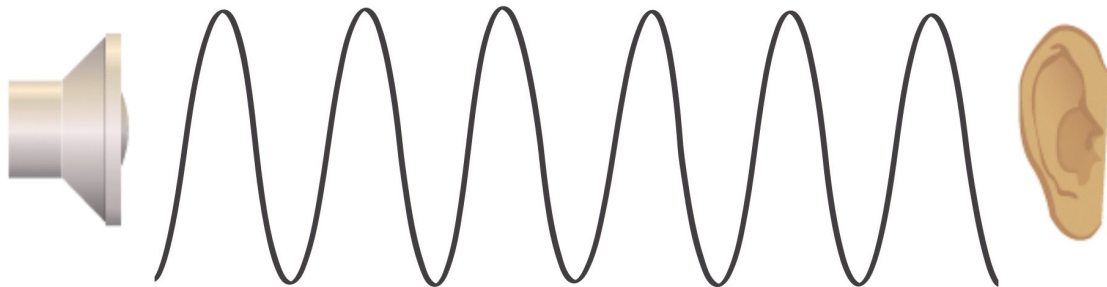


From Sound Waves to Spectrogram

How To Understand Sound Waves?

The raw sound wave mixes everything together - vocals, instruments, all in one wavy line. It's too messy for identification. We need to see the **frequencies** inside the sound.

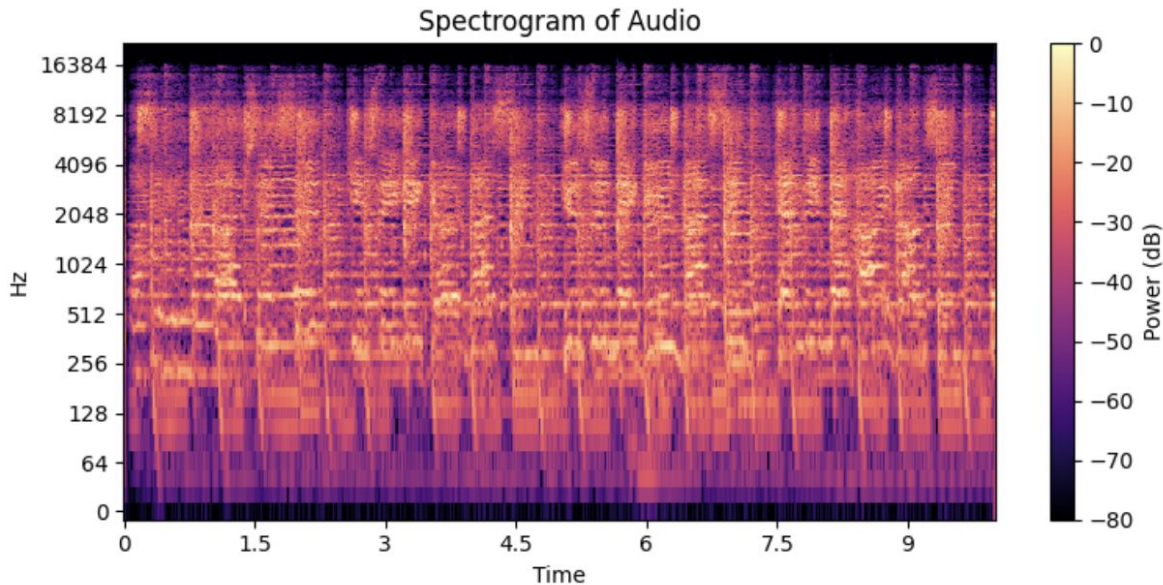
We need to create spectrogram...



From Sound Waves to Spectrogram

Creation of Spectrogram

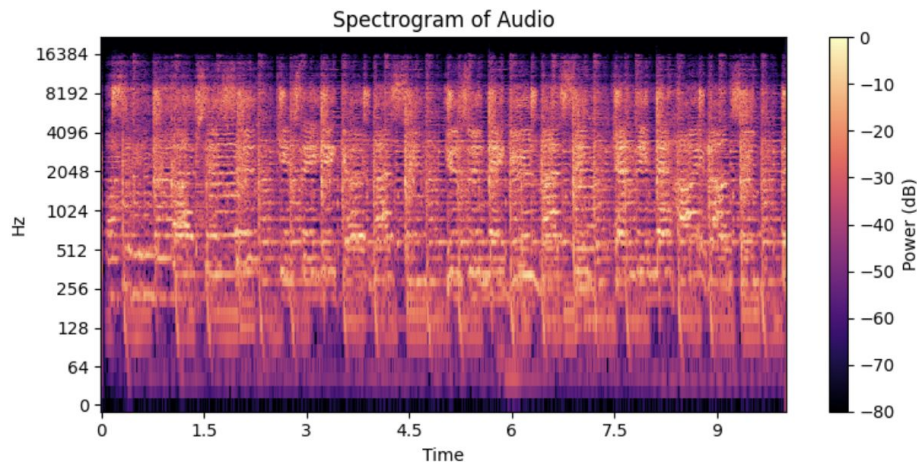
This is my 10-second recording captured **spectrogram** while *Tarkan – Şımarık* was playing.



From Sound Waves to Spectrogram

Spectrogram

- Time is on the **x-axis**
- Frequency (Hz) is on the **y-axis**
- Color shows **power in dB**, brighter = louder
- Each vertical slice shows which frequencies are present at that moment

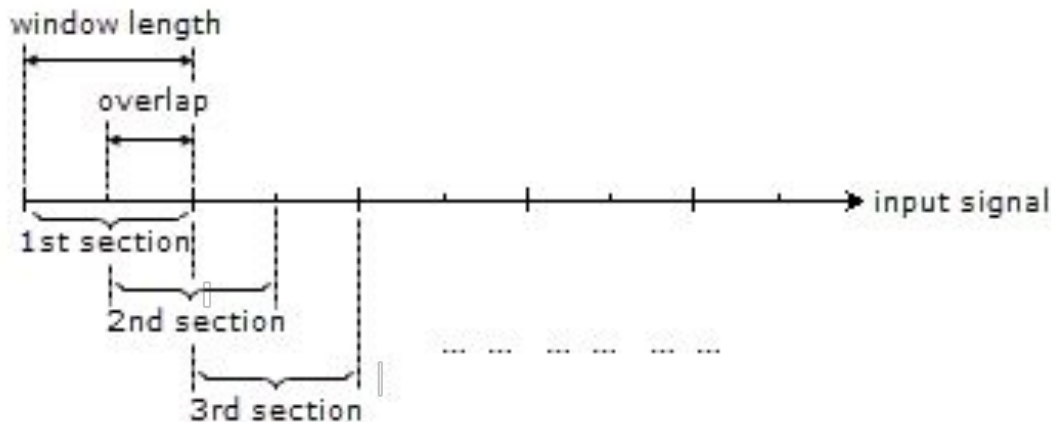


From Sound Waves to Spectrogram

How to Create a Spectrogram?

To create a frequency map, we use the **Short-Time Fourier Transform (STFT)**:

- Split the sound into small overlapping windows
- Apply Fourier Transform to each window
- Shows how frequencies change over time



From Sound Waves to Spectrogram

Creation of Spectrogram with Python

```
import sounddevice as sd
import librosa, librosa.display

fs = 44100
duration = 10
audio = sd.rec(int(duration * fs), samplerate=fs, channels=1, dtype='int16')
sd.wait()
y = audio.flatten()

# Compute STFT and convert to dB
S = np.abs(librosa.stft(y, n_fft=2048, hop_length=512))
S_dB = librosa.amplitude_to_db(S, ref=np.max)

# Plot spectrogram
plt.figure(figsize=(8, 4))
librosa.display.specshow(S_dB, sr=fs, x_axis='time', y_axis='log', cmap='magma')
plt.title("Spectrogram of Audio")
plt.colorbar(label='Power (dB)')
plt.tight_layout()
plt.show()
```

From Sound Waves to Spectrogram

Why **librosa**?



```
pip install librosa
```



Finding What Makes Songs Unique

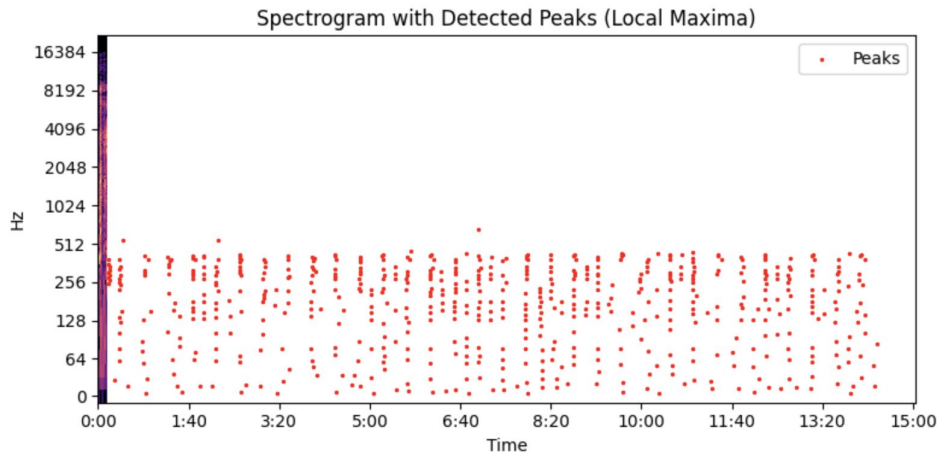
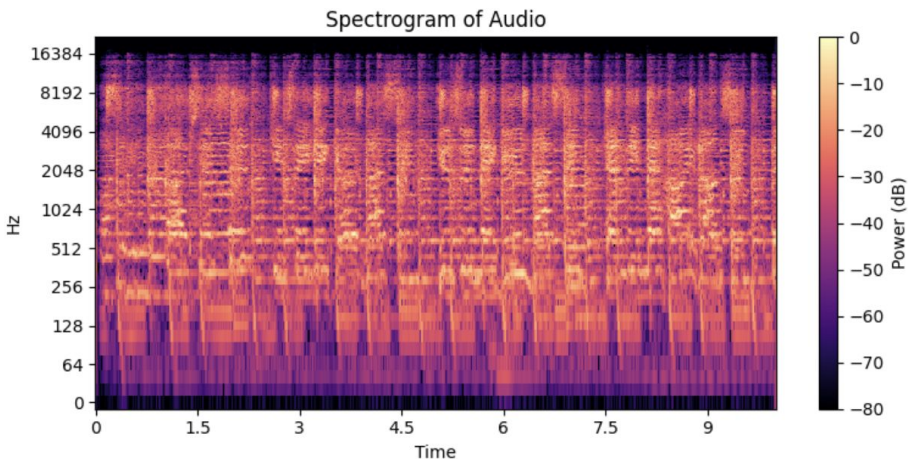
Turning Audio Into a Constellation of Peaks

- After we create the **spectrogram**, we don't need every single point.
- Instead, we find the **most prominent peaks**, points where the sound is strongest.
- Each peak represents a **local maximum** in frequency and time, kind of like a “star” in the night sky.
- This pattern of peaks is called a **constellation map**, it captures the unique acoustic “fingerprint” of the song.

Finding What Makes Songs Unique

From Spectrogram to Constellation of Peaks

- Every song has its own “**constellation**” of peaks.
- Here’s spectrogram with detected peaks while I was playing Tarkan - Şımarık.



Finding What Makes Songs Unique

How to Create Constellation of Peaks?

We find the 'stars' in our audio sky by looking for **local maxima** - points that are brighter (louder) than all their neighbors.

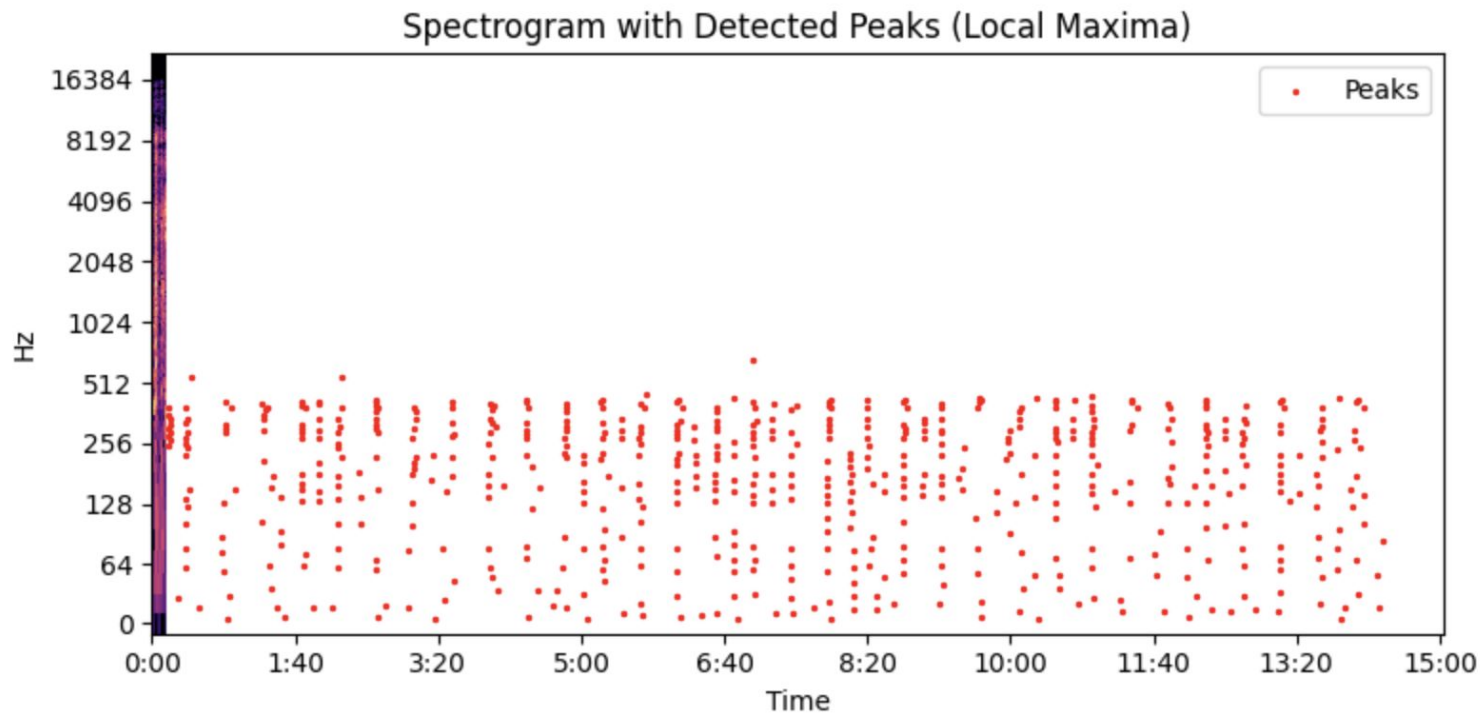
The process:

1. Scan the spectrogram with a sliding window.
2. Find points stronger than their 20x20 frequency-time neighborhood.
3. Keep only the top 10% strongest points.

Result: A unique constellation map of the song's brightest moments.

Finding What Makes Songs Unique

Constellation of Peaks



Finding What Makes Songs Unique

Creation of Constellation of Peaks with Python

```
import numpy as np
import matplotlib.pyplot as plt
import librosa, librosa.display
from scipy.ndimage import maximum_filter

neighborhood_size = (20, 20) # (frequency x time)
local_max = maximum_filter(S, size=neighborhood_size) == S

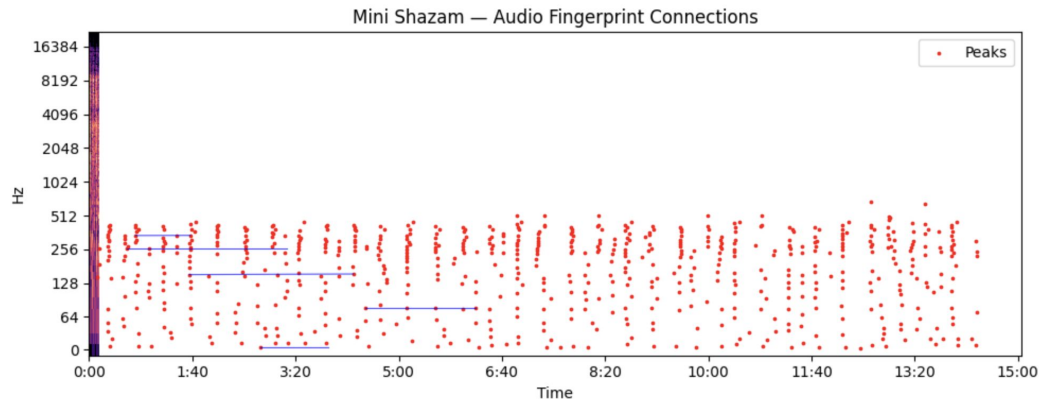
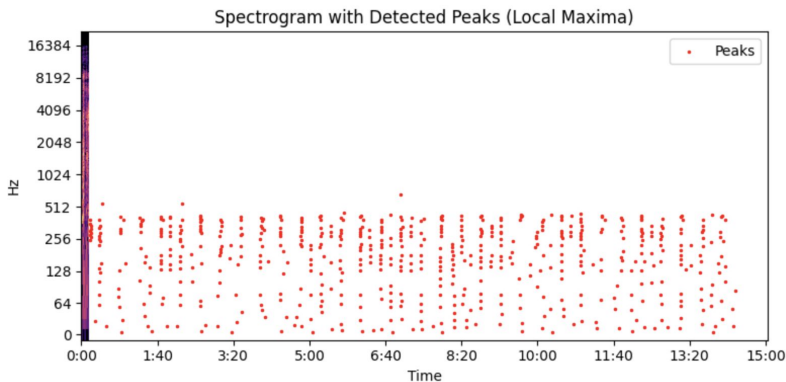
amp_threshold = np.percentile(S, 90)
peaks = local_max & (S > amp_threshold)

plt.figure(figsize=(8, 4))
librosa.display.specshow(S_dB, sr=fs, x_axis='time', y_axis='log', cmap='magma')
plt.scatter(np.where(peaks)[1], np.where(peaks)[0], s=8, c='red', marker='.', label='Peaks')
plt.title("Spectrogram with Detected Peaks (Local Maxima)")
plt.legend()
plt.tight_layout()
plt.show()
```

Building Audio Fingerprints

Turning Constellations into Fingerprints

After we create the constellation map, we need to turn these isolated 'stars' into recognizable 'constellations' that we can actually search for.



Building Audio Fingerprints

How Constellations Become Fingerprints

- Each **peak** (local maximum) represents a strong frequency at a specific time.
- We connect each peak to a few **nearby peaks ahead in time**, forming pairs.
- Each pair is defined by:
 - Two frequencies: (f_1, f_2)
 - Time difference: $\Delta t = t_2 - t_1$
- Together, these form a **unique relationship** that captures the song's structure.
- Each $(f_1, f_2, \Delta t)$ pair is converted into a **compact hash**, this becomes the **audio fingerprint**.

Example Fingerprint:

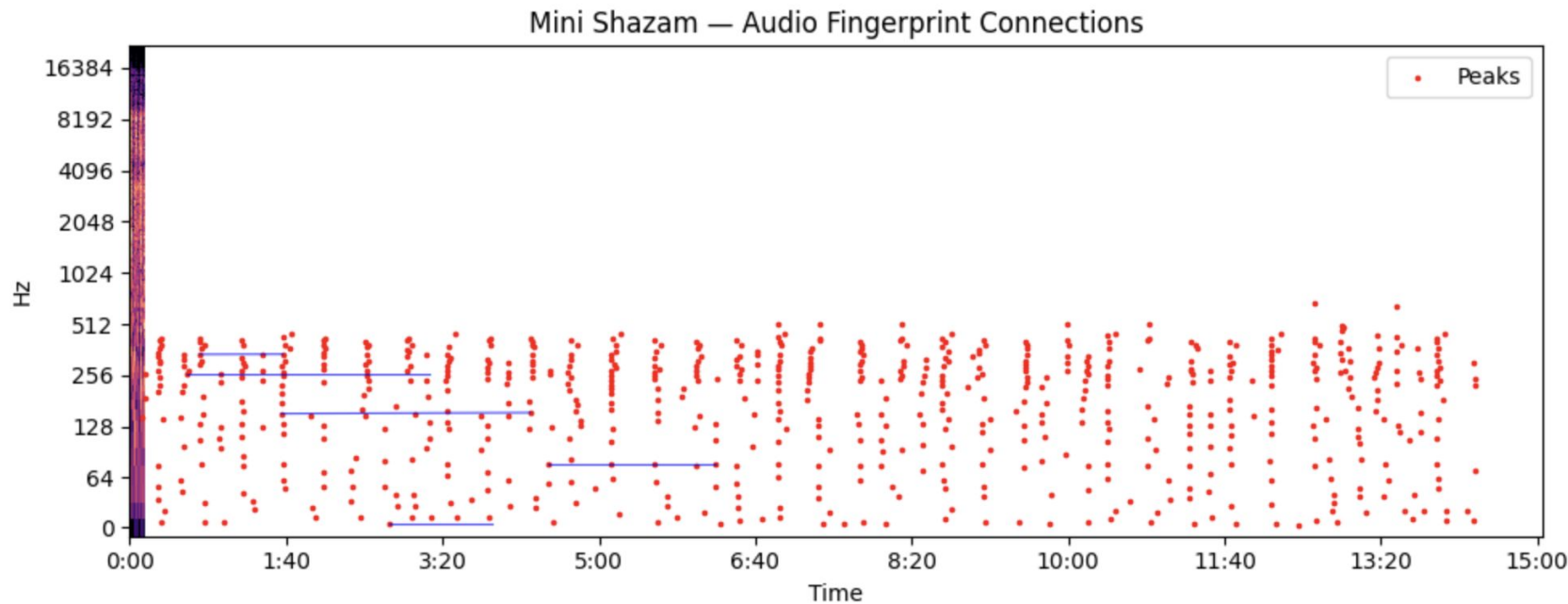
$f_1 = 256 \text{ Hz}$, $f_2 = 1024 \text{ Hz}$, $\Delta t = 1.2 \text{ s}$



Hash $\rightarrow$ "a8c93f..."

Building Audio Fingerprints

Audio Fingerprint Connections



Building Audio Fingerprints

How Constellations Become Fingerprints

```
import hashlib

# Connect spectral peaks to form fingerprint pairs
freqs, times = np.where(peaks)
fingerprints = []
fan_value = 5 # connect each peak to 5 nearby peaks

for i in range(len(times)):
    for j in range(1, fan_value):
        if i + j < len(times):
            f1, f2 = freqs[i], freqs[i + j]
            t1, t2 = times[i], times[i + j]
            delta_t = t2 - t1
            if 0 < delta_t ≤ 200:
                fingerprints.append((f1, f2, delta_t))

# Convert each pair into a compact hash
def hash_fingerprint(f1, f2, dt):
    return hashlib.sha1(f"{f1}|{f2}|{dt}".encode()).hexdigest()[:20]

hashes = [hash_fingerprint(f1, f2, dt) for (f1, f2, dt) in fingerprints]
```

Matching Songs in the Database

Database Structure



- Each song is stored once in the **songs** table with its **title** and **artist**.
- Its audio fingerprints are stored in the **fingerprints** table.

Each fingerprint entry contains:

- the **hash** (unique audio signature),
- the **song ID** it belongs to,
- and its **time offset** in the track.

Matching Songs in the Database

Database Setup



```
# Store song fingerprints in a database
CREATE TABLE songs(id INTEGER PRIMARY KEY, title TEXT, artist TEXT)
CREATE TABLE fingerprints(hash TEXT, song_id INTEGER, offset INTEGER)

# Example insertion
INSERT INTO fingerprints(hash, song_id, offset) VALUES (h, song_id, offset)
```

- The database is created using **SQLite** commands.
- Fingerprints are inserted with their **hash**, **song ID**, and **time offset**.
- During matching, we query the database to find **hash collisions** with incoming fingerprints.

Matching Songs in the Database

Why SQLite?



- Lightweight and easy to set up, no server required.
- Perfect for **local demos or prototypes**.
- Fast for **read-heavy** workloads like fingerprint lookup.
- Stores all data in a **single file**, making it portable.

Matching Songs in the Database

How to Match Songs?

1. **Query & Search:** Take fingerprints from your short audio clip and search for matching hashes in the database.
2. **Calculate Time Difference (Δt):** For every match, calculate the difference between the clip's time and the song's original time.
 - $\Delta t = \text{song_offset} - \text{query_offset}$
3. **Vote Clustering:** Each match casts a "vote" for a specific (song_id, Δt) pair.
4. **Find the Consensus:** The correct song is revealed by the largest cluster of votes for the same Δt .

Matching Songs in the Database

Matching Logic with Python



```
# Align matches based on time offset
votes = {} # (song_id, delta) → count

for (f1, f2, dt, q_offset) in query_fps:
    h = hash_fingerprint(f1, f2, dt)
    cursor.execute("SELECT song_id, offset FROM fingerprints
WHERE hash = ?", (h,))
    for song_id, s_offset in cursor.fetchall():
        delta = int(s_offset) - int(q_offset)
        key = (song_id, delta)
        votes[key] = votes.get(key, 0) + 1
```

Matching Songs in the Database

Finding The Match with Python



```
# Find the song with the highest number of aligned matches
best_match = max(votes.items(), key=lambda x: x[1])
(song_id, delta), score = best_match

print(f"Identified Song ID: {song_id}, Score: {score}, Δt: {delta}")
```

- Each (song_id, Δt) pair gets votes from matching fingerprints
- We find the pair with the maximum votes using max()
- The highest score = strongest time alignment = correct match!

Let's Build It!

A Mini Shazam in Python

Bring them all together!

Create a system that:

- Records a short audio sample
- Extracts its fingerprints
- Compares them to the database
- The song with the highest time-aligned matches (Δt cluster) is identified as the correct track

Mini Shazam

Tap the button to recognize a song



Mini Shazam - Song Recognition App

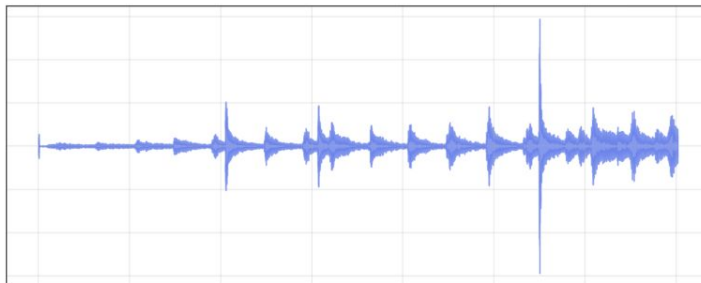


🎤 Listening for 10 seconds...

✅ Recording finished! Recognizing...

Song detected!

Match found: 'Fusion' by Ludwig
Göransson (score=6, delta=42)



A Mini Shazam in Python

Why streamlit?



Streamlit

- Build interactive apps in minutes
- Great for data visualization & ML demos
- Easy to share and deploy (Streamlit Cloud)

Real World Challenges

The World Doesn't Sound Like The Lab

Noise Distortion



Time Offsets



Database Scale



Beyond Music Recognition

Fingerprinting Isn't Just For Music

Speech & Speaker Identification

TV & Ads Tracking

Environmental Sound Detection

Industrial Monitoring

Medical Applications

End of Presentation

Any Questions?

